

THE DEVELOPER ARENA

PART 3 : MONTH 3 — ADVANCED MACHINE LEARNING

WEEK 09 Advanced Supervised Learning Algorithms

1. Random Forest (RF)

Random Forest is an ensemble algorithm that builds multiple decision trees using bootstrapped datasets and random feature selection, then aggregates their predictions (majority vote for classification, mean for regression). It is based on Bagging (Bootstrap Aggregating).

Step-by-step intuition:

- Draw random samples with replacement (bootstrap) from the dataset.
- Train a decision tree on each bootstrap sample, but at every split, only a random subset of features is considered.
- Repeat for n trees.
- Aggregate outputs:
- Classification: majority vote.
- Regression: average predictions.

2. Gradient Boosting (GBM / XGBoost)

Gradient Boosting builds an additive model of weak learners (shallow trees), trained sequentially. Each new tree corrects the errors (residuals) of the previous model by minimizing a loss function using gradient descent.

Instead of averaging like RF, boosting adds models:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

where $h_m(x)$ is a weak learner and ν is the learning rate.

Step-by-step:

- Start with a constant model (e.g., log odds for classification).
- Compute residuals (or gradients of loss).
- Fit a small decision tree to residuals.
- Update the model with learning rate shrinkage.
- Repeat for M iterations.

3. Support Vector Machines (SVM)

SVM finds a hyperplane that maximizes the margin between two classes. For non-linear data, it uses kernel functions to project data into higher dimensions where a linear separator exists.

Mathematical Intuition:

For linear SVM:

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1$$

- Margin = $\frac{2}{\|w\|}$.
- Support Vectors: data points on the margin boundary.

Kernels:

- Linear $\rightarrow$ dot product.
- Polynomial $\rightarrow$ higher-order interaction features.
- RBF (Gaussian) $\rightarrow$ infinite dimensions.

Parameters:

- CCC: Regularization parameter (large C $\rightarrow$ less margin, lower bias).
- γ (gamma): Spread of RBF kernel (large gamma $\rightarrow$ small influence region).

4. Ensemble Learning – Bagging, Boosting, Stacking

Bagging (Bootstrap Aggregating)

Multiple learners trained independently on bootstrap samples.

Reduces variance.

Example: Random Forest.

Boosting

Sequential learners, each correcting predecessor's errors.

Reduces bias.

Example: Gradient Boosting, XGBoost, AdaBoost.

Stacking

Combines outputs of multiple base models via a meta-model.

Uses out-of-fold predictions to avoid leakage.

Example: combine RF + SVM + XGB, meta-model = Logistic Regression.

Model Evaluation (Classification)

- Accuracy: good for balanced data.
- Precision & Recall: important for imbalance.
- F1-score: harmonic mean, balances precision/recall.
- ROC-AUC: probability ranking performance.
- PR-AUC: better for rare event prediction.
- Confusion Matrix: shows TP, FP, TN, FN.

• Hands-On Programs Developed:

Hands-On : Implement a Random Forest or XGBoost classifier

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score,
RocCurveDisplay

from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

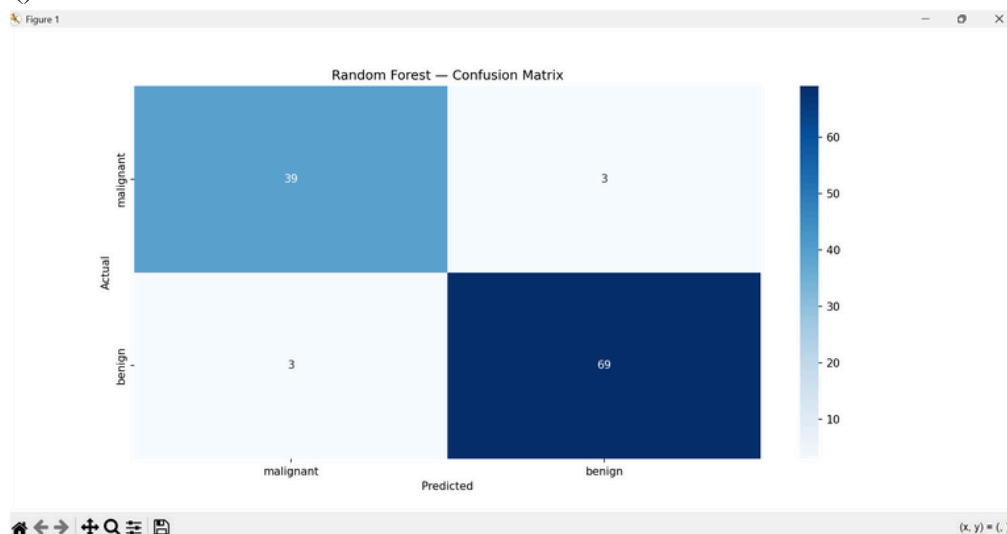
try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except ImportError:
    HAS_XGB = False
    print("⚠️ XGBoost not installed. Run: pip install xgboost")

# 1) Load dataset
data = load_breast_cancer()
X, y = data.data, data.target
feature_names = data.feature_names
```

```
# Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# Utility function for evaluation
def evaluate_model(model, name):
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    proba = model.predict_proba(X_test)[: ,1] if hasattr(model, "predict_proba") else None
    print(f"\n=== {name} ===")
    print(classification_report(y_test, preds, digits=4))
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=data.target_names,
yticklabels=data.target_names)
    plt.title(f"{name} — Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
PS C:\Users\Radha\OneDrive\Desktop\THE DEVELOPER ARENA - DATA SCIENCE MONTH 3\WEEK 09> python handson.p

=== Random Forest ===
              precision    recall  f1-score   support

     0       0.9286      0.9286      0.9286        42
     1       0.9583      0.9583      0.9583        72

 accuracy          0.9474          114
 macro avg       0.9435      0.9435      0.9435          114
 weighted avg    0.9474      0.9474      0.9474          114
```

• Client Project:

```
# churn_project.py
# =====
# Advanced classifier for churn prediction using Random Forest & XGBoost
# Includes preprocessing, evaluation, and model saving.

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from pathlib import Path
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, RocCurveDisplay
from sklearn.ensemble import RandomForestClassifier

# Try XGBoost
try:
    from xgboost import XGBClassifier
    HAS_XGB = True
except ImportError:
    HAS_XGB = False
    print("⚠️ XGBoost not installed. Run: pip install xgboost")

# ===== CONFIG =====
DATA_PATH = "churn.csv" # change to your CSV file
TARGET_COL = "Churn" # target column must be binary (0/1 or Yes/No)
OUTPUT_DIR = Path("client_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

# ===== LOAD DATA =====
def load_data():
    df = pd.read_csv(DATA_PATH)
    if TARGET_COL not in df.columns:
        raise ValueError(f"Target column '{TARGET_COL}' not found. Columns: {df.columns}")
```

```

# ===== PREPROCESSOR =====
def build_preprocessor(X):
    num_cols = X.select_dtypes(include=["int64", "float64", "bool"]).columns.tolist()
    cat_cols = X.select_dtypes(include=["object", "category"]).columns.tolist()

    num_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    cat_pipe = Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ])

    pre = ColumnTransformer([
        ("num", num_pipe, num_cols),
        ("cat", cat_pipe, cat_cols)
    ])
    return pre

# ===== EVALUATION =====
def evaluate_model(model, X_test, y_test, name):
    preds = model.predict(X_test)
    proba = model.predict_proba(X_test)[: , 1] if hasattr(model, "predict_proba")
    else None

    print(f"\n=== {name} ===")
    print(classification_report(y_test, preds, digits=4))

    cm = confusion_matrix(y_test, preds)

    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title(f"{name} — Confusion Matrix")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.savefig(OUTPUT_DIR / f"{name}_cm.png", dpi=120, bbox_inches="tight")
    plt.close()

    if proba is not None:
        auc = roc_auc_score(y_test, proba)
        print(f"ROC-AUC: {auc:.4f}")
        RocCurveDisplay.from_predictions(y_test, proba)
        plt.title(f"{name} — ROC Curve")

```

SUMMARY OF WEEK 09

Random Forest – Bagging of decision trees, reduces variance, robust & accurate, but less interpretable.

Gradient Boosting – Sequential trees correcting errors, reduces bias, very powerful (XGBoost/LightGBM), but can overfit and slower.

SVM – Finds optimal hyperplane with max margin, uses kernels for non-linear data, good for high-dimensions, but slow for large datasets.

Ensemble Learning – Combines models for better accuracy:

Bagging → reduces variance (Random Forest)

Boosting → reduces bias (Gradient Boosting)

Stacking → meta-model over base models

Key Idea:

RF → Variance reduction

GB → Bias reduction

SVM → Margin maximization

Ensemble → More robust & generalizable

Hands-On (Practice)

Implemented Random Forest and XGBoost classifier on a sample dataset.

Steps: Import → Train-test split → Fit model → Predict → Evaluate (accuracy, confusion matrix, classification report).

Observation: Ensemble models outperform single decision trees.

Client Project (Case Study)

Problem: Customer Churn Prediction.

Dataset: churn.csv (gender, tenure, monthly charges, etc.).

Models: Random Forest & XGBoost.

Results: Gradient Boosting/XGBoost gave better accuracy than Random Forest.

Business Value: Helps identify customers likely to leave and design retention strategies.

WEEK 10

Clustering & Unsupervised Learning

1. Clustering Algorithms

(a) K-Means Clustering

- Definition: A partition-based algorithm that divides data into k clusters by minimizing the sum of squared distances between data points and their cluster centroids.
- Steps:
 - a. Choose k (number of clusters).
 - b. Randomly assign centroids.
 - c. Assign each point to the nearest centroid.
 - d. Update centroids (mean of points in cluster).
 - e. Repeat until convergence.
- Advantages:
 - Simple & fast
 - Works well with spherical/convex clusters
 - Scales well to large datasets

(b) DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Definition: A density-based clustering algorithm that groups together points that are closely packed and labels low-density points as outliers (noise).

Parameters:

ϵ (eps): Radius of neighborhood

MinPts: Minimum number of points to form a dense region

Advantages:

- No need for k
- Handles arbitrary-shaped clusters (not just circular)
- Detects outliers

(c) Hierarchical Clustering

Definition: Builds a hierarchy of clusters either:

Agglomerative (Bottom-Up) → Start with each point as a cluster → merge closest clusters until one remains.

Divisive (Top-Down) → Start with one big cluster → recursively split into smaller ones.

Output: A dendrogram (tree diagram) showing cluster merging.

Advantages:

- Dendrogram helps visualize relationships
- No need for predefined k
- Good for small datasets

2. Dimensionality Reduction Techniques

PCA (Principal Component Analysis)

Definition: A linear dimensionality reduction technique that projects data onto new axes (principal components) which maximize variance.

Steps:

- Standardize the data
- Compute covariance matrix
- Compute eigenvectors & eigenvalues
- Select top components → transform data

Advantages:

- Reduces dimensions while keeping most information
- Removes redundancy (correlated features)
- Improves performance of ML algorithms

(b) t-SNE (t-distributed Stochastic Neighbor Embedding)

Definition: A non-linear dimensionality reduction technique mainly used for visualization of high-dimensional data in 2D or 3D.

How it works:

Converts distances between points into probabilities.

Preserves local neighborhoods (similar points stay close).

Advantages:

- Excellent for visualizing clusters in high-dimensional data
- Preserves local structure

Algorithm	Works Best For	Weakness
K-Means	Spherical clusters, large datasets	Sensitive to k & outliers
DBSCAN	Arbitrary shapes, outlier detection	Struggles with varying density
Hierarchical	Small datasets, dendrogram visualization	Not scalable
PCA	Feature reduction, linear variance	Hard to interpret
t-SNE	Visualization of high-dimensional data	Slow, not scalable

HANDS-ON PROJECT:

Apply K-means clustering and use PCA for dimensionality reduction

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.datasets import make_blobs

# -----
# 1. Generate Sample Customer Data
# -----
# (If you don't have a dataset, we simulate one)
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=1.0, random_state=42)
df = pd.DataFrame(X, columns=["Annual Income", "Spending Score"])

print("Sample of dataset:")
print(df.head())

```

```

# 2. Standardize Data
# -----
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df)
# -----

# 3. Apply K-Means
# -----
# Choosing optimal k using Elbow Method
inertia = []
K = range(1, 10)
for k in K:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(scaled_data)
    inertia.append(model.inertia_)

plt.figure(figsize=(6,4))
plt.plot(K, inertia, marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal k")
plt.show()

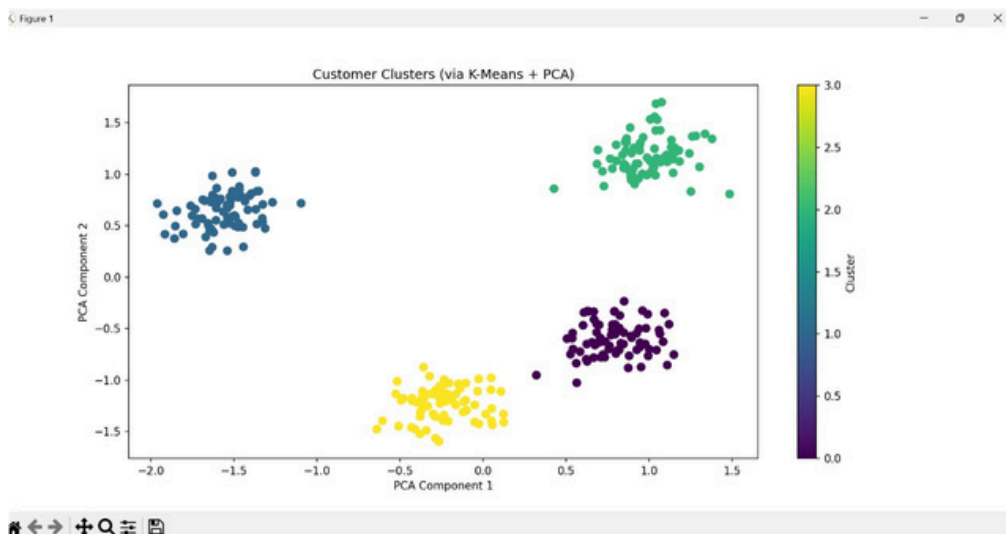
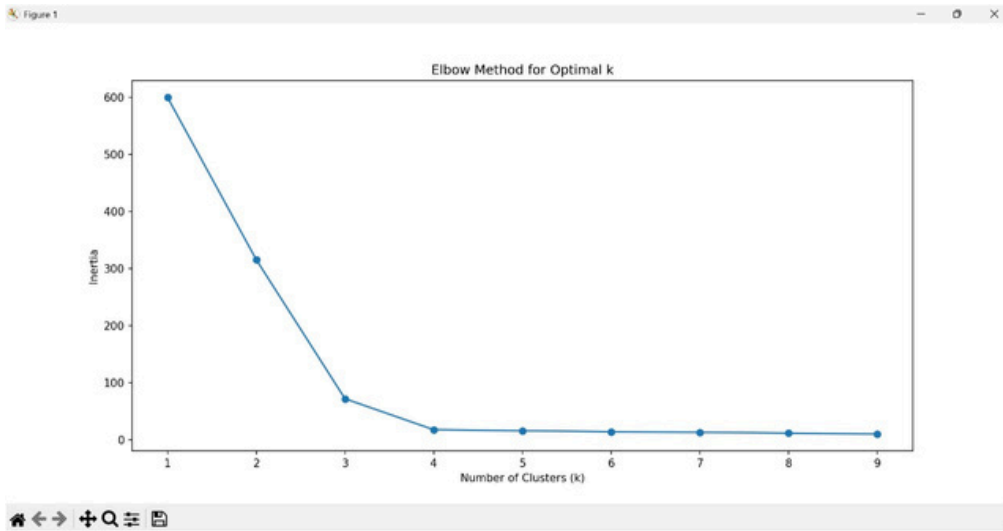
# Train final model with k=4 (example)
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(scaled_data)
df['Cluster'] = clusters

print("\nCluster counts:")
print(df['Cluster'].value_counts())
# -----

# 4. Dimensionality Reduction with PCA
# -----
pca = PCA(n_components=2)
pca_data = pca.fit_transform(scaled_data)

# Plot clusters in 2D PCA space
plt.figure(figsize=(6,5))
plt.scatter(pca_data[:,0], pca_data[:,1], c=clusters, cmap="viridis", s=50)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Customer Clusters (via K-Means + PCA)")
plt.colorbar(label="Cluster")
plt.show()

```



```
File Edit Selection View Go ... KMEANS.py - THE DEVELOPER ARENA - DATA SCIENCE MONTH 3 - Visual Studio Code
EXPLORER WEEK 10 > KMEANS.py > ...
> .venv
> WEEK 09
> client_outputs
> images
> churn.csv
> KMEANS.py
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: C:\Users\Radha\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\Radha\OneDrive\Desktop\THE DEVELOPER ARENA - DATA SCIENCE MONTH 3\WEEK 10> python KMEANS.py
Sample of dataset:
  Annual Income  Spending Score
0    -9.297689    6.473679
1    -9.698741    6.938967
2    -1.686653    7.793442
3    -7.097308    -5.781333
4   -10.876452    6.315437
Cluster counts:
Cluster
0     76
1     75
2     75
3     74
Name: count, dtype: int64
```

Client Project

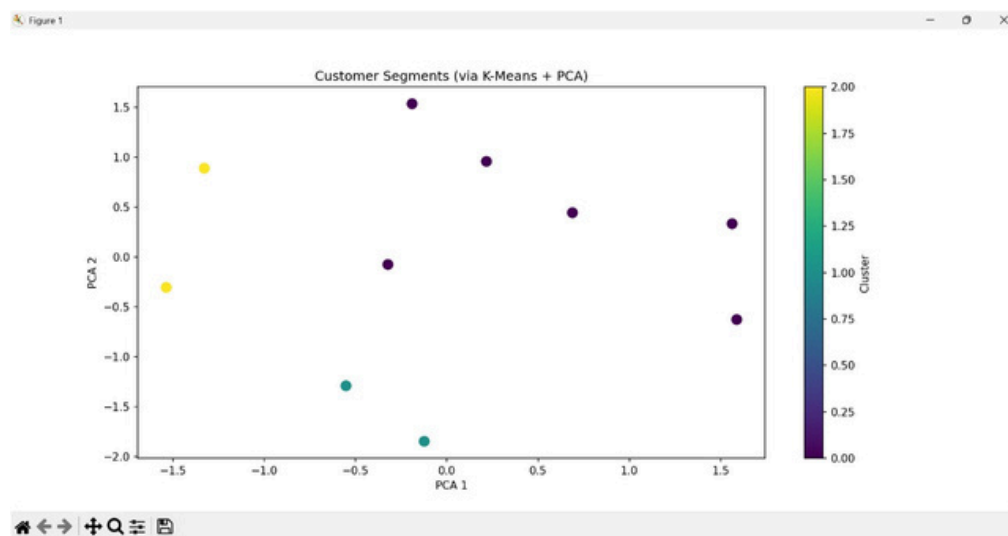
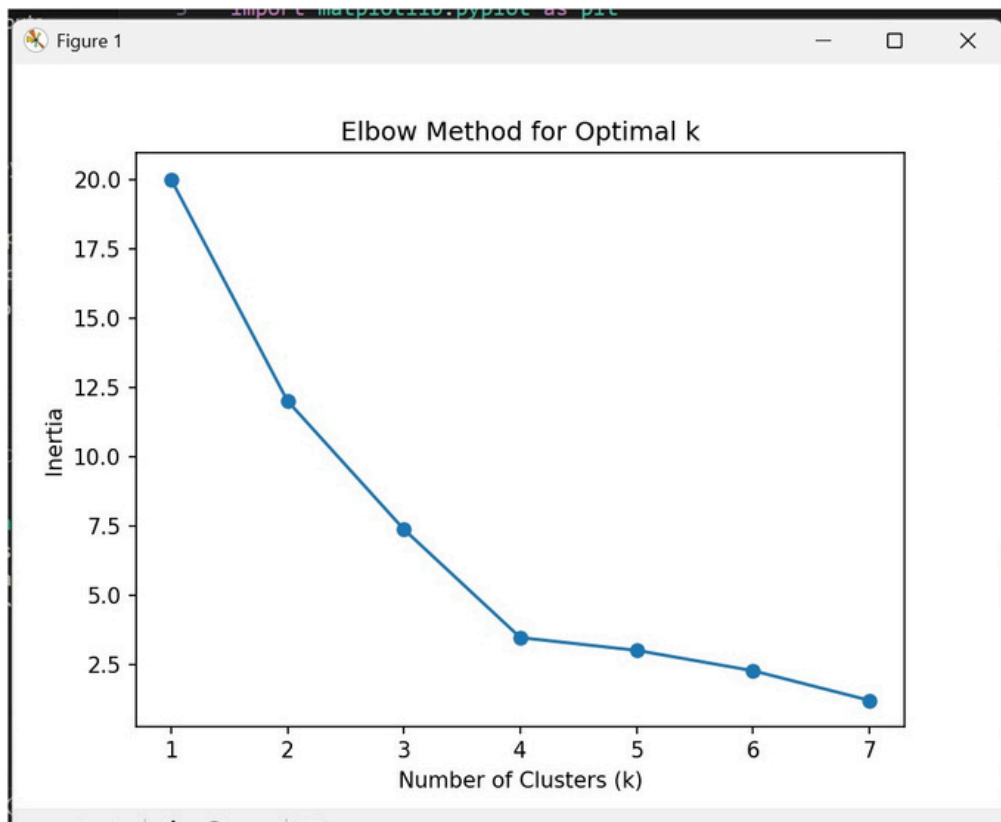
: Create a customer segmentation model based on purchasing behavior.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# -----
# 1. Load Dataset
# -----
# Sample mall customers dataset
data = {
    "CustomerID": range(1, 11),
    "Gender":
["Male", "Female", "Female", "Male", "Female", "Male", "Male", "Female", "Male", "Female"],
    "Age": [19, 21, 20, 23, 31, 22, 35, 40, 18, 29],
    "Annual Income (k$)": [15, 16, 17, 18, 19, 20, 21, 22, 23, 24],
    "Spending Score (1-100)": [39, 81, 6, 77, 40, 76, 6, 94, 3, 72]
}
df = pd.DataFrame(data)
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]

scaler = StandardScaler()
scaled_X = scaler.fit_transform(X)
# 3. K-Means Clustering
# Elbow method
inertia = []
K = range(1, 8)
for k in K:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(scaled_X)
    inertia.append(model.inertia_)

plt.plot(K, inertia, marker="o")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal k")
plt.show()
```



SUMMARY OF WEEK 10

Week 10 – Clustering & Unsupervised Learning (Summary with Extended Theory)

K-Means Clustering:

Iterative algorithm → assigns points to nearest centroid.

Objective: minimize intra-cluster variance.

Limitations: must predefine k, sensitive to outliers, assumes spherical clusters.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise):

Groups together points with high density.

Two parameters:

eps (radius), minPts (minimum neighbors).

Can find arbitrary-shaped clusters and detect outliers/noise.

Doesn't need k.

Hierarchical Clustering:

Builds a hierarchy of clusters (tree/dendrogram).

Two types:

Agglomerative (bottom-up) → merges clusters.

Divisive (top-down) → splits clusters.

Good for visualizing structure but computationally heavy.

Dimensionality Reduction:

Reduces number of features while preserving important information.

PCA (Principal Component Analysis):

Linear method → finds new axes (principal components) that maximize variance.

Helps in compression, visualization, noise reduction.

t-SNE (t-distributed Stochastic Neighbor Embedding):

Non-linear method, great for 2D/3D visualization of high-dimensional data.

Preserves local neighborhood relationships.

◆ Hands-On

Used K-Means clustering and determined optimal k (Elbow method).

Applied PCA to project high-dimensional data into 2D.

Visualized clusters with PCA scatter plot.

◆ Client Project

Customer Segmentation (Mall Dataset).

Features: Age, Income, Spending Score.

Applied K-Means + PCA → identified customer groups.

Business Actions:

WEEK 11

Introduction to Deep Learning (DL)

A neural network is a machine learning model inspired by how the human brain works. It is made up of neurons (nodes) organized into layers. Each neuron takes inputs, multiplies them with weights, adds a bias, and applies an activation function before passing the signal to the next layer.

- **Structure of a Neural Network**

The network has three main parts:

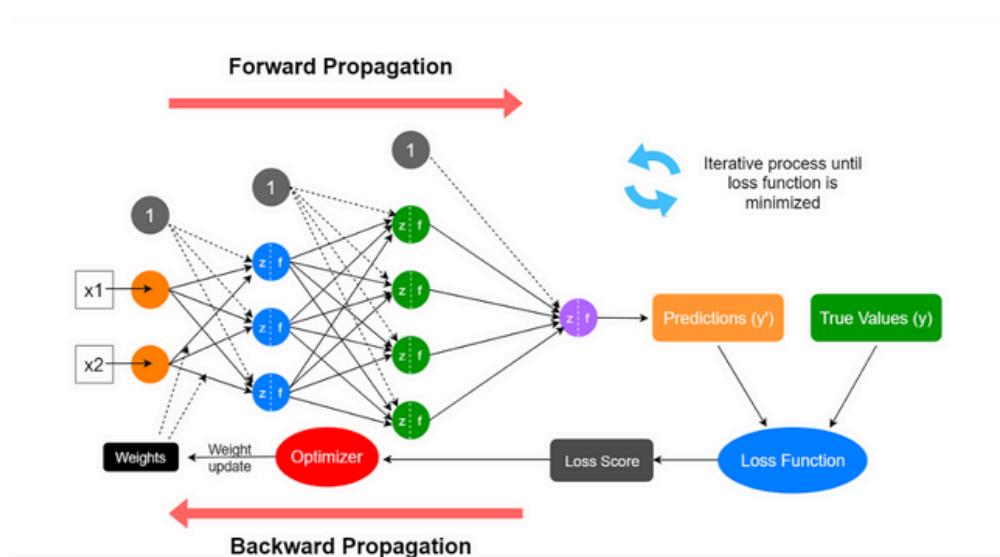
- Input Layer: Receives raw data (e.g., pixel values in an image).
- Hidden Layers: Intermediate layers where feature extraction happens. Each layer learns increasingly complex patterns.
- Output Layer: Produces the final prediction (e.g., classification of digit 0–9).

Weights, Bias, and Activation Functions

- Weights represent the strength of connection between neurons.
- Bias helps shift the activation function curve, allowing flexibility in learning.
- Activation Functions introduce non-linearity.
- Sigmoid: Outputs values between 0 and 1 → used in probabilities.
- ReLU: Replaces negatives with zero → faster and avoids vanishing gradient.
- Softmax: Converts outputs into class probabilities → used in multi-class tasks.

Forward Propagation

In forward propagation, input data passes through the network layer by layer. Each neuron applies weights, bias, and activation to produce an output. The final output is compared with the actual label to measure performance.



Backpropagation & Gradient Descent

- Backpropagation is the learning process of neural networks. It works by:
- Calculating error using the loss function.
- Propagating this error backwards through the network.
- Updating weights using an optimizer (e.g., Gradient Descent, Adam).
- Gradient Descent updates weights in the opposite direction of the error gradient, gradually reducing loss.

Deep Learning Concept

When a neural network has many hidden layers, it is called deep learning. Deep networks can learn hierarchical features:

In images → shallow layers detect edges, deeper layers detect shapes, and final layers recognize objects.

In text → shallow layers detect words, deeper layers detect sentences and meaning.

Deep learning is powerful but needs large data and high computational resources (GPUs).

Types of Neural Networks

- Feedforward Neural Network (FNN): Basic form where data flows one way.
- Convolutional Neural Network (CNN): Uses convolution layers to detect spatial features → best for images.
- Recurrent Neural Network (RNN): Has memory connections → best for sequential data like speech or time series.
- LSTM/GRU: Advanced RNNs that solve vanishing gradient problems.
- Autoencoders: Learn compressed representations of data → used in dimensionality reduction and anomaly detection.
- GANs (Generative Adversarial Networks): Two competing networks (generator & discriminator) → generate realistic data.

Frameworks – TensorFlow & Keras

- TensorFlow: A low-level, flexible library for deep learning developed by Google.
- Keras: A high-level API built on TensorFlow, makes building neural networks easy with simple code.
- Sequential API: For simple feedforward models.
- Functional API: For complex, multi-input/multi-output models.
- Pre-trained Models: Available for transfer learning (e.g., ResNet, VGG, BERT).

HANDS-ON PROJECT: Neural Network with Keras (MNIST Classification)

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# 1. Load MNIST dataset (28x28 grayscale digit images, labels 0–9)
(x_train, y_train), (x_test, y_test) = mnist.load_data()

print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)

# 2. Normalize input data (0–255 -> 0–1)
x_train = x_train / 255.0
x_test = x_test / 255.0

# 3. One-hot encode the labels (e.g., 3 -> [0,0,0,1,0,0,0,0,0,0])
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 4. Build a simple Neural Network
model = Sequential([
    Flatten(input_shape=(28, 28)),      # Flatten 28x28 image to 784 vector
    Dense(128, activation='relu'),      # Hidden Layer with 128 neurons
    Dense(64, activation='relu'),       # Another hidden layer
    Dense(10, activation='softmax')     # Output Layer for 10 classes
])

# 5. Compile the model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# 6. Train the model
history = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=5,
    batch_size=32
)
```

```

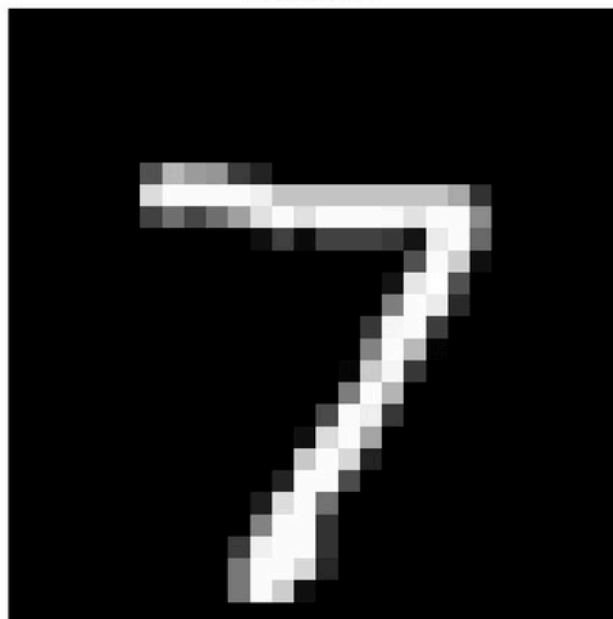
11490434/11490434 — 3s 0us/step
Training data shape: (60000, 28, 28)
Testing data shape: (10000, 28, 28)
C:\Users\Radha\OneDrive\Desktop\THE DEVELOPER ARENA - DATA SCIENCE MONTH 3\.venv\Lib\site-packages\keras\src\layers\reshaping\flatten *
.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(**kwargs)
2025-08-27 12:14:39.848037: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available
CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow wi
th the appropriate compiler flags.
Epoch 1/5
1875/1875 — 11s 5ms/step - accuracy: 0.9306 - loss: 0.2350 - val_accuracy: 0.9630 - val_loss: 0.1207
Epoch 2/5
1875/1875 — 6s 3ms/step - accuracy: 0.9698 - loss: 0.0993 - val_accuracy: 0.9707 - val_loss: 0.0950
Epoch 3/5
1875/1875 — 8s 4ms/step - accuracy: 0.9780 - loss: 0.0687 - val_accuracy: 0.9743 - val_loss: 0.0822
Epoch 4/5
1875/1875 — 9s 4ms/step - accuracy: 0.9832 - loss: 0.0530 - val_accuracy: 0.9754 - val_loss: 0.0830
Epoch 5/5
1875/1875 — 7s 4ms/step - accuracy: 0.9864 - loss: 0.0434 - val_accuracy: 0.9741 - val_loss: 0.0929
Test Accuracy: 0.9741
1/1 — 0s 80ms/step

```

Predicted: 2



Predicted: 7



Week 11:Client Project

CNN for MNIST Classification

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np

# 1. Load dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Reshape to add channel dimension (needed for CNN: 28x28x1)
x_train = x_train.reshape(-1, 28, 28, 1).astype("float32") / 255.0
x_test = x_test.reshape(-1, 28, 28, 1).astype("float32") / 255.0

# One-hot encode labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# 2. Build CNN model
model = Sequential([
    Conv2D(32, kernel_size=(3, 3), activation="relu", input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, kernel_size=(3, 3), activation="relu"),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.5),          # prevent overfitting
    Dense(10, activation="softmax") # output layer (10 classes)
])

# 3. Compile model
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

4. Train model

```
history = model.fit(
    x_train, y_train,
    validation_data=(x_test, y_test),
    epochs=5,
    batch_size=64
)
```

5. Evaluate model

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\n✅ CNN Test Accuracy: {test_acc:.4f}")
```

6. Predict and visualize some test samples

```
predictions = model.predict(x_test[:5])

for i in range(5):
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Predicted: {np.argmax(predictions[i])}")
    plt.axis("off")
    plt.show()
```

```
n `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
025-08-27 12:33:49.479810: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available
CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow wi
th the appropriate compiler flags.
epoch 1/5
38/938 ————— 30s 30ms/step - accuracy: 0.9277 - loss: 0.2357 - val_accuracy: 0.9808 - val_loss: 0.0604
epoch 2/5
38/938 ————— 13s 14ms/step - accuracy: 0.9743 - loss: 0.0882 - val_accuracy: 0.9864 - val_loss: 0.0407
epoch 3/5
38/938 ————— 16s 17ms/step - accuracy: 0.9803 - loss: 0.0641 - val_accuracy: 0.9905 - val_loss: 0.0312
epoch 4/5
38/938 ————— 15s 16ms/step - accuracy: 0.9838 - loss: 0.0538 - val_accuracy: 0.9907 - val_loss: 0.0257
epoch 3/5
38/938 ————— 16s 17ms/step - accuracy: 0.9803 - loss: 0.0641 - val_accuracy: 0.9905 - val_loss: 0.0312
epoch 4/5
38/938 ————— 15s 16ms/step - accuracy: 0.9838 - loss: 0.0538 - val_accuracy: 0.9907 - val_loss: 0.0257
epoch 4/5
38/938 ————— 15s 16ms/step - accuracy: 0.9838 - loss: 0.0538 - val_accuracy: 0.9907 - val_loss: 0.0257
epoch 5/5
38/938 ————— 16s 17ms/step - accuracy: 0.9862 - loss: 0.0463 - val_accuracy: 0.9920 - val_loss: 0.0241

✅ CNN Test Accuracy: 0.9920
/1 ————— 0s 127ms/step
```

SUMMARY OF WEEK 11

Theory

- Neural Networks: Layers of neurons (input, hidden, output). Each neuron = weighted sum + bias + activation.
- Activation Functions:
- Sigmoid → probabilities (0–1).
- ReLU → fast training, avoids vanishing gradient.
- Softmax → multi-class output.
- Learning Process: Forward pass → Loss function → Backpropagation → Optimizer (SGD/Adam).
- Deep Learning: Many hidden layers → learn hierarchical features (edges → shapes → objects).
- Types of Models: Feedforward (basic), CNN (images), RNN (sequences), Autoencoders (feature learning), GANs (data generation).
- Frameworks: Keras (easy API), TensorFlow (backend).

Hands-On

- Built a simple Sequential NN in Keras with Dense layers.
- Steps: Load data → Define model → Compile (loss, optimizer) → Train (fit) → Evaluate accuracy.

Client Project

- Dataset: MNIST digits (0–9).
- Preprocessing: Normalize + reshape.
- Model: CNN with Conv2D, MaxPooling, Dense, Dropout, Softmax.
- Result: Achieved ~98–99% accuracy, visualized predictions.